VII MAINTENANCE

Action Summary

Management should establish policies, standards, and procedures for maintaining systems and components that include detailed roles and responsibilities. It is important that policies and procedures effectively ensure the availability and continued operability of systems and components.

Examiners should review the following:

- System and component inventory adequacy for key information (e.g., to facilitate the entity identifying and addressing vulnerabilities).
- Maintenance plans, including maintenance schedules and logs, vendor and developer recommendations, cost-benefit analyses, operational risk assessments, and qualified staffing adequacy.
- Change management processes, including change types, authorizations, and related risk assessments.
- Security monitoring reports (e.g., anomalous activity).
- Configuration management practices to validate appropriate implementation and enforcement of established change processes.
- Vulnerability and threat identification processes, including remediation plans.
- Any maintenance-related reports to the board of directors and senior management.
- IT asset inventory, including considerations for EOL, such as planned obsolescence and planning for upgrades to legacy systems and components.
- Termination, disposal, and off-boarding processes of systems, components, and change in third-party relationships.
- Maintenance documentation for any system, component, and configuration updates.

The maintenance[237] and operations[238] SDLC activities are necessary whether systems and components have been developed internally or by a third party. They include the routine servicing and periodic modification of everything needed for a system or component to run correctly including the network, hardware, system software, databases, and the related documentation. An entity's maintenance policies and procedures should include maintenance roles and responsibilities (which may include third-party roles and responsibilities), maintenance access policy (internal and remote), and maintenance monitoring and auditing mechanisms.

[237] As noted in the FFIEC Glossary, "maintenance" is "any act that either prevents the failure or malfunction of equipment or restores its operating capability."

[238] As noted in the FFIEC Glossary, "operations" are "the performance of activities comprising methods, principles, processes, procedures, and services that support business functions." IT operations include the tactical management of technology assets and daily delivery of services to capture, transmit, process, and store transactions and information that support the entity's overall business processes.

Well-managed modifications can improve a system's or component's functionality, confidentiality, integrity, availability, and resilience. Periodic modifications may address user functional requirements, correct security vulnerabilities, enhance performance, or increase storage.

An accurate and complete IT asset inventory, including system and component maintenance, is central to ITAM. For example, an inventory that contains system and component versions and patch levels allows an entity to quickly determine how a newly discovered vulnerability affects the organization. Another use of the IT asset inventory is to determine which systems are affected by components reaching their EOL. IT asset inventories should include IoT products.[239] For more information on ITAM, refer to the *FFIEC IT Handbook's* "Architecture, Infrastructure, and Operations" booklet.

VII.A Preventive Maintenance

Preventive maintenance is the activity to proactively address situations that may cause issues or disruptions. For example, an entity may review a system's storage utilization periodically to determine whether storage is adequate to prevent related outages.

Management should establish a preventive maintenance plan, especially for mission-critical systems and components. When developing the preventive maintenance schedule, it is important to consider system or component criticality, developer maintenance recommendations, operational risks, and other relevant factors (e.g., emerging technologies and threats). These maintenance plans should be aligned with business plans. Appropriate entity personnel schedule maintenance activities during time frames that minimize the risk of interruption to business processes such as after business hours, during weekends, and during holidays.

System and component access for maintenance activities should be limited to only that part of the system needed to perform the maintenance and only for the time necessary to conduct the maintenance activity. Capturing and reviewing maintenance activity logs minimizes the risk that malicious activity occurs during a maintenance window.

For more information, refer to the *FFIEC IT Handbook's* "Architecture, Infrastructure, and Operations" booklet.

VII.B Change Management

Change management encompasses the planning, governance (including approvals), testing, and implementation of any form of change. Examples of changes that are important to manage are configuration changes, software patches, and functional changes.

A key piece of change management is change control, further explained in the "Implementing Changes" section of this booklet. Changes may be necessary for a variety of reasons, including

[239] For more information, refer to NIST IR 8425, *Profile of the IoT Core Baseline for Consumer IoT Products*, and NIST IR 8259A, *IoT Device Cybersecurity Capability Core Baseline*.

updates or modifications to systems or components, requests from users, and results from vulnerability scans or penetration tests. The speed with which these changes should be implemented varies. For example, configuration changes or software patches that address a newly discovered vulnerability in a critical system should be completed quickly, while changes to improve user functionality in a low criticality system may have a slower implementation time frame.

The complexity and, in some cases, the system's or component's age may determine the type and frequency of changes needed to maintain the system or component. A system that is complex and contains multiple components may need to be changed more frequently than a simple system that is less reliant on multiple components. Similarly, a system or component that is new may experience more changes when it is initially placed into a production environment versus a system or component that has existed in a production environment for many years. Additionally, systems and components used by large groups may involve more frequent changes to meet the needs of a greater number of users.

Management should be aware of and manage the risks associated with unauthorized, untested, and unplanned changes. These risks include breach of confidentiality, breach of integrity (e.g., accidental configuration errors), lack of availability (e.g., system failure resulting from an inappropriate change or not implementing a change), or lack of resilience. Appropriate processes should be followed to manage changes (e.g., configuration management, vulnerability management, and patch management) regardless of whether the system or component was developed internally.

- **Configuration management:** Configuration management policies, standards, and procedures for developed and procured systems and components should be established, and validation performed to ensure that they are followed. The policies, standards, and procedures should include the identification and the use of system and component baseline configurations[240] (or original versions). The policies, standards, and procedures should specify that management evaluates, approves, documents, and disseminates changes to baseline configurations. Automated tools may simplify and promote consistency throughout the process. As previously stated, effective management helps ensure that changes to the configuration follow an established change management process. For more information, refer to the *FFIEC IT Handbook's* "Information Security" and "Architecture, Infrastructure, and Operations" booklets.
- **Vulnerability management:** Sources of vulnerability information (e.g., including threat intelligence) for the entity's systems and components should be identified. These sources may include the vendor, supplier, or developer, public-private information-sharing partnerships (e.g., the Financial Services Information Sharing and Analysis Center [FS-

[240] NIST defines "baseline configuration" as "the documented set of specifications for a system, or a configuration item within a system, that has been formally reviewed and agreed on at a given point in time, and which can be changed only through change control procedures."

ISAC][241] and NIST National Vulnerability Database[242]), and internal and external testing. Entity personnel should remediate relevant vulnerabilities and address threats continually and efficiently. The criticality of the identified vulnerability and the affected system or component can dictate the timing of the remediation. Appropriate senior leadership should review vulnerability management results, such as whether the most severe vulnerabilities are remediated within appropriate time frames, as specified in policy. Open or unresolved vulnerabilities in an entity's systems and components could result in loss of confidentiality, integrity, availability, and resilience.

- **Patch management:** Patch management processes are a critical part of an entity's effective change management practices. The entity's patch management processes should include procedures for identifying, evaluating, approving, testing, installing, and documenting patches or updates for systems and components. Regardless of whether the systems or components are developed internally or by a third party, it is critical to have a process for the timing, prioritization, testing, and deployment of patches to minimize the potential of vulnerability exploits that may result in system compromise or service disruption. Reasons for not applying patches should be documented and periodically reviewed by appropriate personnel, independent of the decision to not apply the patch. These processes help mitigate the risk that unaddressed vulnerabilities exist in systems and components and lead to system compromise. For more information, refer to the *FFIEC IT Handbook's* "Information Security," "Architecture, Infrastructure, and Operations," and "Audit" booklets.

VII.B.1 Implementing Changes

Action Summary

Management should have a process for the request, decision, approval or rejection, test, and implementation of changes. The goal of the change process should always be system or component confidentiality, integrity, availability, and resilience.

Examiners should review the following:

- Change management policies and procedures.
- Change logs.
- Relevant access controls.
- Processes for controlling system and software versions.
- Sample change documentation (e.g., change requests and approvals), including risk assessments.
- Change management program training.
- Conversion plans and coordination with conversion partners and stakeholders.

[241] FS-ISAC is an industry consortium dedicated to reducing cyber risk in the global financial system.

[242] NIST's National Vulnerability Database is the U.S. government's repository of standards-based vulnerability management data represented using the Security Content Automation Protocol. These data enable automation of vulnerability management, security measurement, and compliance. The database includes security checklist references, security-related software flaws, misconfigurations, product names, and impact metrics.

The process of implementing changes is sometimes referred to as change control. This process helps ensure that IT-related modifications are appropriately authorized, tested, documented, implemented, and monitored. Effective change control processes help management ensure that patches, updates, and configuration changes will not adversely affect a system, component, or application on the network. The characteristics and risks of a system, activity, or change should dictate the formality of the change control process. Management should maintain policies, standards, and procedures that specify the change control process, including defined roles and responsibilities. Examples of key roles with associated responsibilities are the project sponsor, change manager, quality manager, information security manager, auditor, network and system administrators, user support, and users.

A change manager is responsible for managing all changes affecting the entity to realize the intended improvements with minimal or no disruptions to operations. A group of stakeholders can aid the change manager in the assessment, prioritization, and scheduling of changes. Examples of this type of group include change advisory boards and change control review boards, which often consist of representatives from all areas in the entity's IT department, affected business lines, and third parties. This individual or group should develop communication protocols to ensure that all affected stakeholders are notified of changes.

Management should prioritize and categorize changes. Changes may be prioritized based on entity requirements (e.g., IT and information security); resources required; and legal, regulatory, and contractual reasons for the change. Change requests may be categorized as rejected, approved, or closed. Prioritizing focuses the organization on completion of the most important changes first. Categorizing changes helps the entity monitor change management performance. Therefore, appropriate change performance metrics and reports should be established.

Effective management has a method to track and report changes (e.g., standard change request forms, library and version controls, and spreadsheets or automated change logs). A change control tracking and reporting process, which may be manual or automated, gives management information about the number, priority, and status of change requests, allowing management to make informed decisions and adjustments as necessary. Maintaining and reviewing detailed change logs is critical for any change control processes. The absence of sound controls and adequate documentation of implemented changes can cause problems when designated personnel install subsequent system changes. Without adequate documentation and controls, personnel will not have the necessary information and may make a change that has an adverse effect on operations.

Effective management follows a defined change control process to make routine and planned changes to systems or components, adjust configuration settings, and make changes to remediate flaws. Management should also account for emergency and unscheduled changes in the change control process. When planning the change, effective management prepares for unexpected problems or failures with the change by creating a back-out plan and documenting the steps taken during the change in the order they occurred. This allows management to back out of a change partially if not all systems and components are affected. Generally, a change control process consists of defined steps, which may address the following:

- **Request:** The change request should include details of the change (e.g., system or component to be changed, description of the change, and justification); impact analysis to identify security needs, risks, and affected systems; change time frame; and back-out plan.
- **Review:** Stakeholders review change requests to determine their viability and business practicality.[243] If reviewers approve a change request, they also prioritize it based on the system's criticality, type of change, interdependencies and interconnections with the involved system, and duration of the change (i.e., time the involved system may be offline or degraded).
- **Approval:** An appropriate level of management, sometimes in collaboration with personnel or a committee, should determine approval of change decisions. Documentation of the decision and approval depends on the type of change initiated. Entity personnel should decide on changes in a timely manner (i.e., in advance of the change). Standard processes may not be followed in emergency situations. However, emergency changes should still be documented, even if after the fact. Emergency change processes should include a list of personnel approved to authorize emergency changes. All changes should be captured through the change control tracking and reporting process.
- **Design and build:** The developer (both internal and external) designs and builds the change to fulfill a request. When completed, the developer provides the requested change to the team or individual responsible for testing.
- **Test:** The team tests the proposed change independently for security, functionality, and any potential adverse effects of the change. Testing should verify that the change performs as intended, identifies any flaws (e.g., integrity issues), and that the change integrates with other systems as intended. Patches and updates that are deployed without testing may have unintended consequences (e.g., deployment delays and operational disruptions) and result in change rollback. Testing should be documented so that testing policy and procedure implementation may be audited.
- **Implementation:** Before deploying a change, an implementation plan (i.e., steps to deploy the change) should be created, full copy of the current production version backed up, and the back-out plan finalized. The team should deploy the approved change according to the implementation plan, preferably during off-peak hours or planned system downtime to minimize the system's or component's time offline.
- **Verify and close:** After implementation, management should perform a post-implementation review to verify that the change was deployed according to the implementation plan and functions appropriately. A comparison of modified programs with change authorization documents serves to validate that only approved changes were implemented. After verification, effective management follows processes to document completion of the change and close the change request. After closing, management should report on the status of the change and monitor the implemented change for unintended issues.

Security is critical when implementing changes for different systems or functions, such as network and client-server environments, mainframes, operating and application programs, and development and procurement projects. To help ensure that modifications do not adversely affect the security posture of varying systems, effective management integrates security into its change

[243] The review may be performed by IT management and stakeholders (e.g., system owner, technical staff, or financial personnel). The reviewers may form a formal committee (e.g., IT steering or operations) depending on the proposed change's scope, costs, risks, impact, and implementation time frame.

control processes. Failure to include information security considerations when implementing changes can result in operational disruptions, degradation in a system's performance, or inappropriate security controls in the system. A change impact analysis can help entity personnel determine the extent to which a change to a system or component may affect its operational, security, and compliance posture. Inaccurately assessing a change's effect could increase the likelihood of an operational disruption or a threat exploiting a vulnerability at the entity.

An impact analysis is completed before the entity makes a final decision regarding the change in the case of a planned change, or after implementation in the case of emergency changes. For a significant change, post-implementation reviews should identify any material impacts that were not anticipated, especially if there were any adverse effects on confidentiality, integrity, availability, resilience, or compliance.

VII.B.2 Additional Control Considerations in Change Management

VII.B.2(a) *Data Controls in the Testing Environment*

Use of production data in test environments should be accompanied by appropriate controls. Some entities have a testing environment that allows the testing of patches, updates, and fixes in a nonproduction environment that is similar or identical to the production environment, including the same configurations and data sets. Use of a test environment that matches the production environment raises the probability that testing will be effective in identifying flaws. If production data is copied into the test environment, it should be protected as if it were in the production environment. Another option is for an entity to generate synthetic data[244] for use in the test environment instead of using production data. Additionally, the testing environment should be isolated and otherwise protected to avoid it being a vulnerable exploit vector.

VII.B.2(b) *Library Controls*

Effective management strictly controls the movement of programs and files among development, quality management, and production libraries[245] for purposes of change control. Libraries allow programmers to access frequently used routines[246] and add them to programs without having to rewrite code. Libraries include executable code modules that run as part of larger applications. Inappropriate segregation of duties between development staff and non-development staff who manage the libraries can lead to production system defects, or the introduction of malicious programs, which in turn can negatively affect system confidentiality, integrity, availability, or resilience.

[244] According to NIST Glossary, "synthetic data generation" is "a process in which seed data are used to create artificial data that have some of the statistical characteristics of the seed data."

[245] In computing, the term "libraries" refers to collections of stored documentation, programs, and data typically segregated by the type of stored information, such as development, testing, and production-related programs, data, or documentation, and arranged for quick identification and reuse. Program libraries include reusable program routines or modules stored in source or object code formats.

[246] For purposes of this discussion, a "routine" is defined as "a sequence of computer instructions for performing a particular task."

Assigning librarian functions to independent personnel (e.g., quality management personnel in more complex entities or non-operations personnel in less complex entities) reduces the risk of errant or malicious code being moved to production environments. Additionally, independent program integrity verification, library activity logs, and documented approval processes also mitigate this risk. These controls should ensure that code in libraries is not altered or deleted. Additional library controls, to mitigate these risks include the following:

- **Object grouping and associated access controls:** Rather than establishing controls on individual objects in libraries, which can create security administration burden, entity personnel or automated programs group similar objects (e.g., executable and nonexecutable routines, test data, and production data) into libraries based on object type. Access can then be granted at library levels.
- **Role-based, automated library applications:** When feasible, the implementation of role-based, automated library applications can help management by restricting access at library or object levels. Additionally, such applications can produce reports that identify who accessed a library and what, if any, changes were made.

Effective management considers using these automated change controls commensurate with the complexity of the entity's IT environment and the number, types, and complexities of changes in that environment. Regardless of whether the entity has automated change control tools, management should strictly control access to production software libraries, particularly in distributed environments.

VII.B.2(c) *Code Repository Controls*

Code[247] can be stored in repositories.[248] They are used in the process of development and configuration management. Through version-control features, code repositories help management track and control different versions of developed source code and program documentation during development, as well as deployment scripts and configuration files. Code repositories may be hosted in-house, on third-party platforms, or on publicly available collaboration venues.

Management should establish code repository use policies. The policies should specify which code repositories the entity uses and where code repository data actually resides, especially for any cloud-based repositories (e.g., public, private, and community clouds). Effective policies and practices include the following actions:

- Prohibiting, when appropriate, the use of any public-based forums, especially if using a private account for entity business or data use.
- Establishing source code ownership.

[247] Code types include source code, executable code, and configuration-as-code.

[248] Repositories are locations where assets may be stored in no certain order. Repositories may contain assets such as libraries, data, and code.

- Requiring business account use to manage entity repository data and correspondingly prohibiting personal account use.
- Prohibiting entity credential use (e.g., email address, password) when creating personal repository accounts.
- Controlling and tracking entity accounts for personnel if cloud repositories are authorized.
- Prohibiting posting entity-owned code in open (i.e., public) forums.
- Evaluating security and other repository risks before using them.
    - Using available security measures, such as MFA.
    - Auditing to determine that all repository-suggested practices, including security measures, are appropriately adopted.
    - Ensuring sanitization of data, if applicable.
    - Restricting exfiltration of code to personal accounts.
- Implementing technical or logical controls (e.g., data loss prevention filters) to block sensitive data from being published to unapproved repositories.
- Searching open (i.e., public) code repositories for entity-owned source code for unauthorized posting of company-owned source code.

Code repositories can be used to manage software changes. For example, code repositories often have built-in version control functionality. Version control tools can be used to manage both software code and associated documentation. Lack of version control can result in updates to the wrong code version and inappropriate access to code. Additionally, inconsistency in code location and naming convention can result in development and maintenance inefficiency. Therefore, available version control features should be used to track changes made to the code and to increase individual accountability for changes.[249]

Another important security control for code repositories is access control. To prevent unauthorized access to or modification of code, access to code repositories should be role-based, follow the principle of least privilege, and limit access to authorized personnel, tools, and services. Management may consider authorization strategies (e.g., zero trust) to mitigate risk. The following are examples of effective access controls:

- Request and approval processes for access to code repositories (e.g., development, staging, or production).
- MFA requirement for access to all (internal and cloud-based) code repositories.
- Appropriate segregation of duties to prevent developer access to the staging and production code repositories, prevent quality management personnel from having access to production code repositories, and grant access to production code repositories only to release management personnel.
- Periodic review processes for access roles and repository logs.

[249] Refer to NIST SP 800-218, *Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities*.

The entity's continuity planning should include code repository backup to facilitate recovery when the development, staging, or production environments are disrupted. Additional information related to the principles and concepts discussed in this section is available in the *FFIEC IT Handbook's* "Architecture, Infrastructure, and Operations," "Information Security," "Management," and "Audit" booklets.

VII.B.3 Change Types

There are several change or modification types that can be controlled through use of a defined change process. The change types are routine or planned modifications, major modifications, and emergency modifications. Effective management has procedures for changes that include change request, review, and approval that direct management to plan, test, and document changes before implementation. Regardless of the change type, management should ensure that changes to any IT system, component, or service are supported by an orderly, adaptable, documented, and measurable process to promote the consistent implementation of changes as well as provide an audit trail for changes. This gives management the necessary information for resilience purposes (e.g., specific rebuild point) or if there are problems when implementing the change (e.g., specific back-out procedures).

VII.B.3(a) *Routine Modifications*

Routine or planned modifications include changes to systems or components to improve performance, add functionality, enhance usability, address defects, improve security, or comply with a new law or regulation. Sometimes referred to as standard changes, routine modifications can be simple or complex but are not considered major and generally can be implemented during the normal course of business. Routine modifications follow repeatable procedures and are often implemented with automation. They include patches, version updates, and firewall changes. They also may include replacing or upgrading network hardware (e.g., printers, user computers, and networking devices). Defined implementation plans, which often include using automated deployment tools to deploy the modifications, are important especially when changes are implemented over numerous systems or components, or across widely dispersed networks.

Effective management plans and coordinates routine modifications because IT changes often affect multiple business lines. Change management discussions should involve sufficient representation from lines of business, IT, information security, quality management, and audit. It is important for involved personnel to receive training to ensure that changes support entity objectives and do not adversely affect confidentiality, integrity, availability, and resilience. Centralized oversight (e.g., through a committee) helps to manage the interdependencies of the entity's systems and operations. Committees providing oversight help clarify request requirements set by the entity and help ensure that all departments are aware of pending changes. Change management discussions to coordinate change activities may happen using specialized change control committees (e.g., CCB) or less formally through management discussions (e.g., during IT steering committee meetings).

After completing a routine modification, all systems and components should be backed up to an off-site location, secured, and maintained to ensure adequate confidentiality, integrity,